Comprehensive Analysis: HackItFinTech

📌 Project Overview

HackItFinTech is a full-stack financial technology application that features a **React-based frontend** and a **Python (FastAPI) backend**. It serves as an investment forecasting and financial learning platform. The application uses Machine Learning (Linear Regression) combined with historical market data to predict future asset prices, while adjusting for real-world economic factors like interest rates and inflation.

The platform provides users with multiple financial tools, including dashboards, asset forecasting, portfolio analysis, educational modules, and chatbots.

🛠️ Technology Stack

Backend Stack

- **Core Framework**: Python, FastAPI
- **Data Manipulation & Processing**: Pandas, NumPy
- **Machine Learning**: Scikit-Learn (`LinearRegression`)
- **Data Source / Finance API**: `yfinance` (Yahoo Finance API)
- **Model Serialization**: `joblib`
- **Server**: `uvicorn` (ASGI server)

Frontend Stack

- **Core Framework**: React 19, Vite
- **Styling**: Tailwind CSS (v4), PostCSS
- **Animations**: Framer Motion
- **Data Visualization**: Recharts
- **HTTP Client**: Axios
- **Routing**: React Router (`react-router-dom`)
- **Icons**: Lucide React

Infrastructure / Dev Tools

- **Concurrency**: `concurrently` is used via Node.js scripts to run both the FastAPI server and the Vite dev server simultaneously.

🏗️ Architecture & Component Deep Dive

1. The Backend (`/asset-backend`)

The backend acts as the data ingestion layer and the predictive engine.

- **Data Fetching (`data_fetch.py`)**
  - Iterates over multiple asset tickers (e.g., Apple stock `AAPL` , Bitcoin `BTC-USD` , Ethereum `ETH-USD` , Gold `GC=F` , Crude Oil `CL=F`).
  - Downloads the last 10 years of historical close prices using `yfinance` .
  - Calculates engineered features: **30-Day Moving Average** and **Volatility** (percentage change standard deviation).
  - Saves the structured data into CSV files (e.g., `AAPL_data.csv`).

- **Model Training (`model_train.py`)**
  - Loads the generated CSV datasets.
  - Cleans the data by dropping empty values and standardizes numeric columns.
  - Re-engineers target variables (the "Future Price").
  - Trains a **Linear Regression** model using a train/test split. Feature inputs include `Close` , `30D_MA` , `Volatility` , and `Price Change` .
  - Determines the Mean Absolute Error to gauge metric accuracy and saves the trained model to disk as a `.pkl` file (e.g., `AAPL_model.pkl`) using `joblib` .
- **API Application Layer (`app.py`)**
  - Exposes a `POST /predict/` FastAPI endpoint.
  - Expects user inputs containing the asset details, interest rate, inflation rate, and forecast timeline (years).
  - It pulls real-time (last 5 years) data on the fly.
  - Feeds the recent stats into the ML model (`AAPL_model.pkl`) to get a base prediction.
  - **Dynamic Adjustments**: Calculates an expected compound return and tweaks the final price using the user-provided inflation and interest rates.
  - Allows Cross-Origin requests (CORS) explicitly for the frontend running at `localhost:5173` .

2. The Frontend (`/frontend`)

The frontend is a modern and highly interactive Web SPA (Single Page Application) that communicates heavily with the backend.

- **Routing & Structure**: Contains a modularly split structure inside `/src` :
  - `LandingPage` : The introductory marketing/hero landing screen.
  - `LoginPage` : Authentication flow UI.
  - `Dashboard` : The central user portal and hub.
  - `AssetForecast` & `StocksAnalysis` : Provides interfaces to interact with backend ML predictions and view real-time projections over adjustable timelines.
  - `CalculationTools` : General calculators for yield, compound interest, or amortizations.
  - `LearningModules` : Educational components to teach users financial literacy.
  - `FloatingChatbot` , `ScamPreventionBot` , `StockBot` : AI-driven conversational UI components providing users with assistance, security warnings, and market contexts.
- **Visuals**: Uses **Tailwind CSS** to manage responsive and modern designs without leaving the JS module. **Framer Motion** enables interactive micro-animations and smooth page transitions.
- **Charts**: Interactive graphs visualizing historical curves and projected trajectories are powered by **Recharts**.

🚀 Execution & Developer Experience

The project uses a mono-repo-like root `package.json` aiming to simplify the execution process. By running `npm start` , a developer kicks off a `concurrently` cycle which spins up:

1. `npm run backend` : `uvicorn app:app --reload --port 8000` inside `asset-backend`
2. `npm run frontend` : `vite` inside `frontend`

There is also an `install-all` script specifically configured to orchestrate both `npm install` and `pip install` across necessary sub-directories, reducing the friction in bootstrapping the local development environment.